EMA for RNNS
In the Context of
Time Series Data

Contents

- Practical Overview of RNN Forms
- Dataset Overview and Context
- Problems Encountered
- Results (Quantitative Noise vs Qualitative Observations)
- Simple Architecture – (Biggest Failure Point of Simple RNNs)

Practical Overview of RNN Forms

Positives

- Computationally cheap - (relative to transformers due to requiring fewer parameters)
- Good at capturing local sequential patterns
- Good at dealing with input of varying length - (due to inputs being Iterative (or even *recurring*)

Negatives

- Training can be volatile/ inconsistent due to
  - Exploding gradients
  - Vanishing gradients
- Sequential, so potentially slow
- Can have difficulty detecting some trends if small or extreme
  - (More of a specific model and optimizer issue)

Considerations and aspects you need to work around

- Window size and prediction target
- Can your test/validation fit your window size?
- Sequence to one?
- Sequence to many?
- Batch size, Generalisation vs Stability

Alternatives - LSTM

- Has a longer term memory state called a cell state “C”
- Has 3 activation layer “gates” to decide how information is propagated
  - **Forget gate** to remove information (sigmoid)
  - **Input gate** to decide what is written to C (sigmoid/tanh)
  - **Output gate** to decide what is written to h

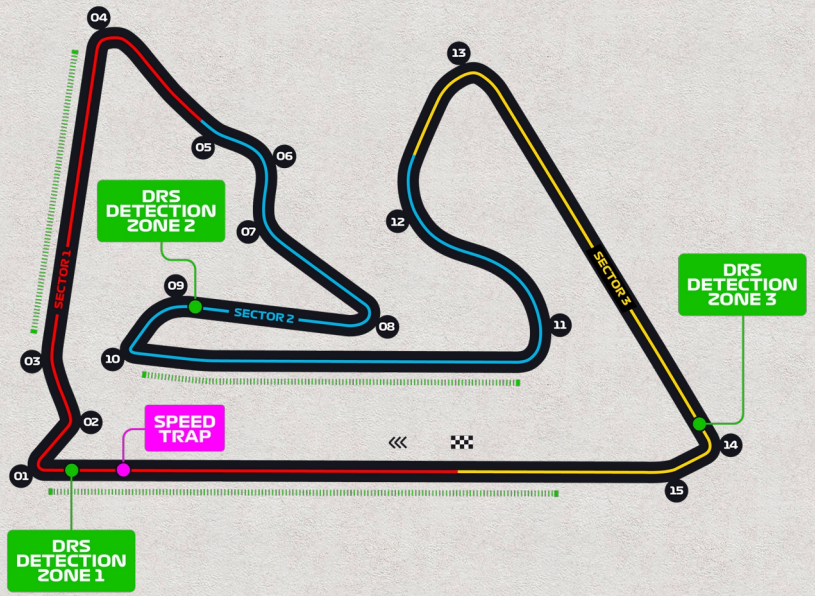
Alternatives -GRU

- An attempt to Simplify LSTM whilst maintaining performance
- Update Gate (mix of forget and input from LSTM) focuses on what is worth keeping
- Reset gate focuses on purging redundant information

Datasets

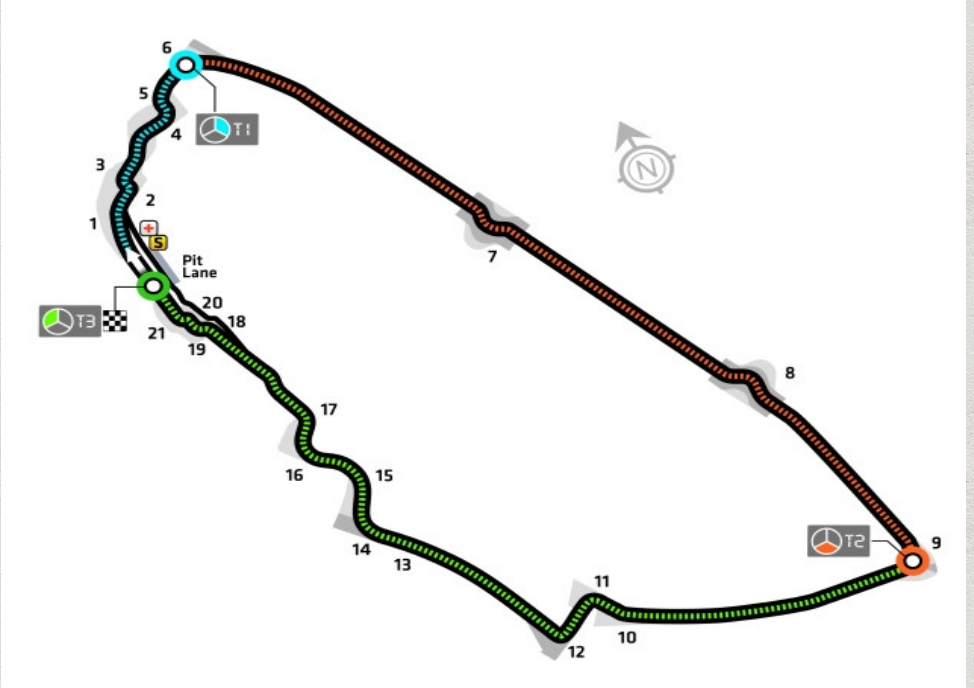
Dataset 1 : F1 (Bahrain)

- Obtained through fastf1 API
- Very feature rich
- Limited in terms of continuous runtime
- Practice sessions do not behave identically to race sessions
- Variety of samplings attempted:
  - Multiple Years (2022-2025) of Bahrain was the final selection



Dataset 2 : Le Mans

- Obtained via Kaggle
- Not feature rich
- Continuous runtime not a serious issue *
- No practice data
- Years 2021 and 2022



Problem #0 – Difficult to Replicate Problems

Volatility in training can lead to:

- A problem
- The problem being solved simply by rerunning the fitting
- No problem
- A different, yet somehow equally difficult to replicate problem

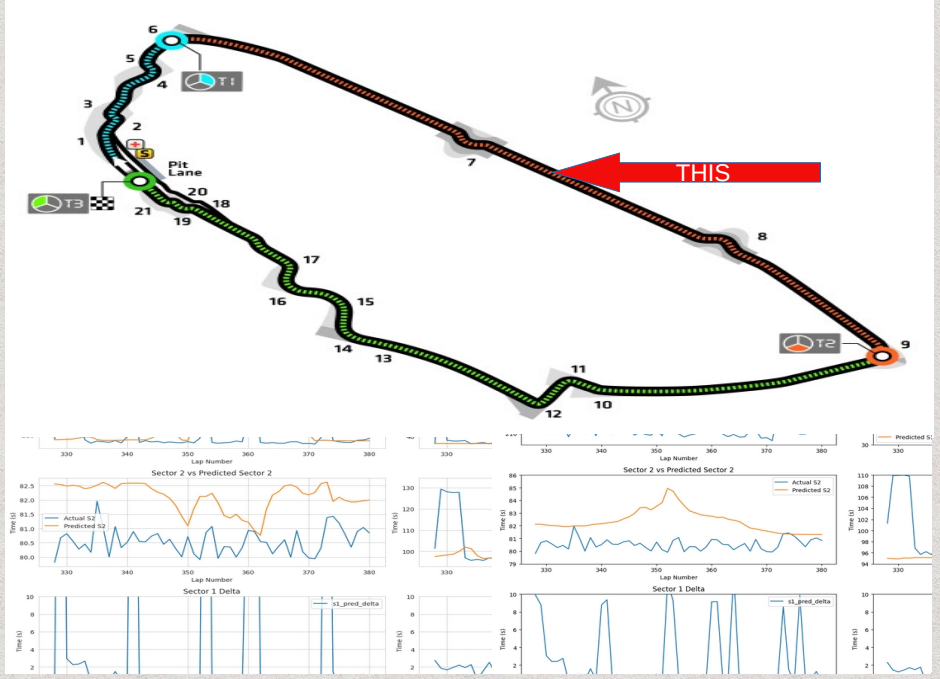
Problem #1 – Average collapse

- Poor noise to signal ratio leads to loss functions playing it safe by defaulting to an average
- Not to be confused with trends with small gradients

s1_pred
33.414024
33.414024
33.413975
33.413990
33.413956
33.414043
33.414204
33.414326
33.413857
33.414040
33.414356
33.414356
33.414352
33.414349
33.414352
33.414364

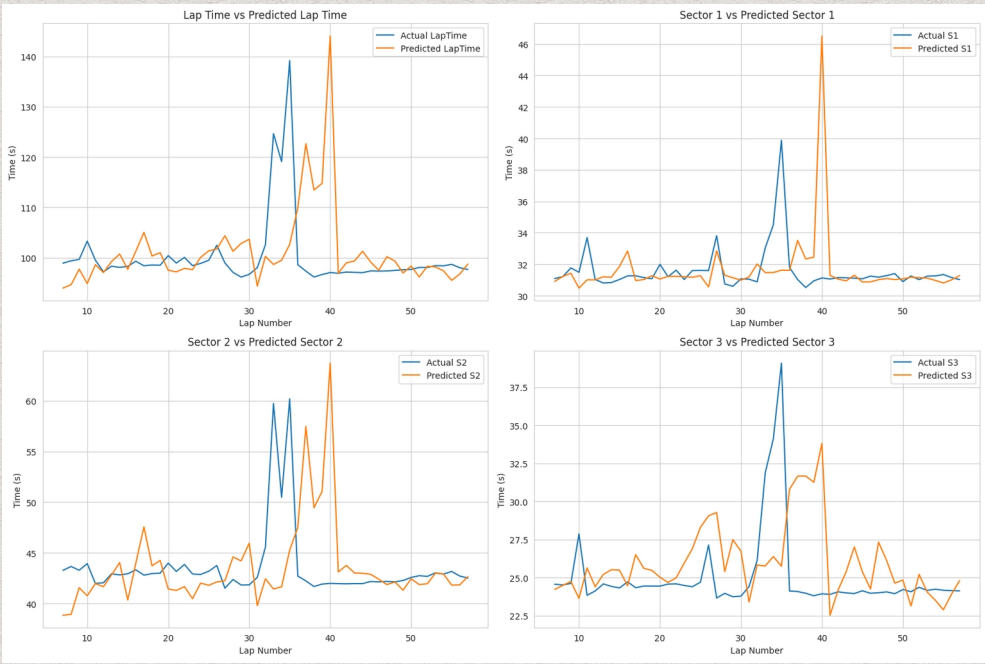
Problem #2 Circuit de la Sarthe (Le Mans) Sector 2

- One of the causes:
  - not seeing past real data in the test set.
- Sector 2 is particularly vulnerable as it has the least variance.
  - The model is not (just) struggling to track variance, it is struggling to find the “base” value.



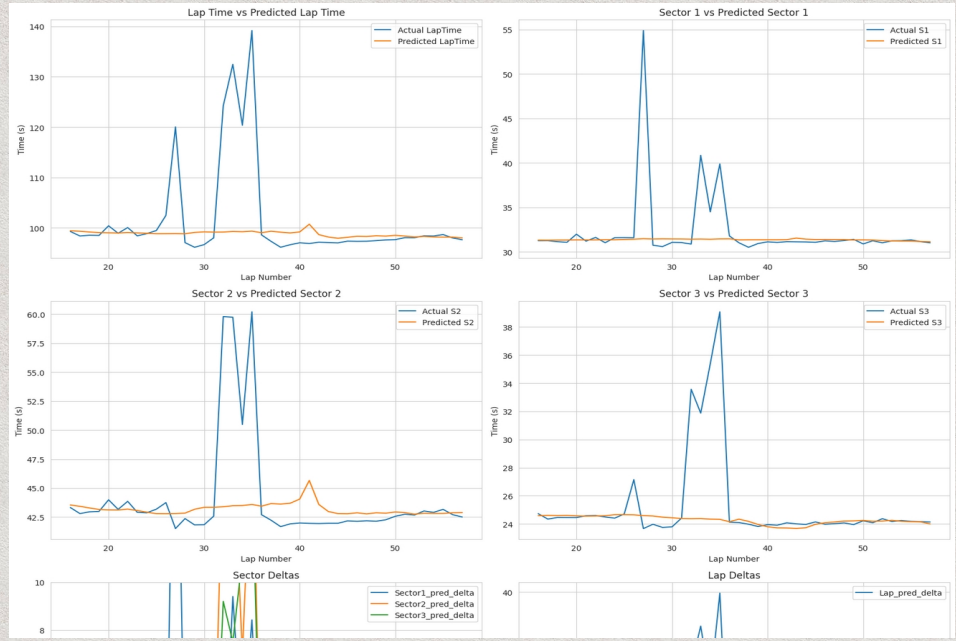
Problem #3
Effective Receptive Field vs Sequence length

- Ghosts of data past are held in the convolution layers
- ERF = (Sum of kernel sizes) – (number of convolution layers)
- Rule of thumb:
  - ERF to sequence length ratio = 0.5 is a good start
  - ERF to sequence length ratio = 0.25 is more reactive



Problem #3
Effective Receptive Field vs Sequence length

- Ghosts of data past are held in the convolution layers
- ERF = (Sum of kernel sizes) – (number of convolution layers)
- Rule of thumb:
  - ERF to sequence length ratio = 0.5 is a good start
  - ERF to sequence length ratio = 0.25 is more reactive



RNN/CNN Hybrid:
SimpleRNN vs LMST vs GRU layers

with Transformer and TSMixer as Controls / Baselines

Le Mans :
Sequence length = 24
Batch size = 32

F1 (Bahrain) :
Sequence length = 14
Batch size = 32

Huber Loss

Le Mans

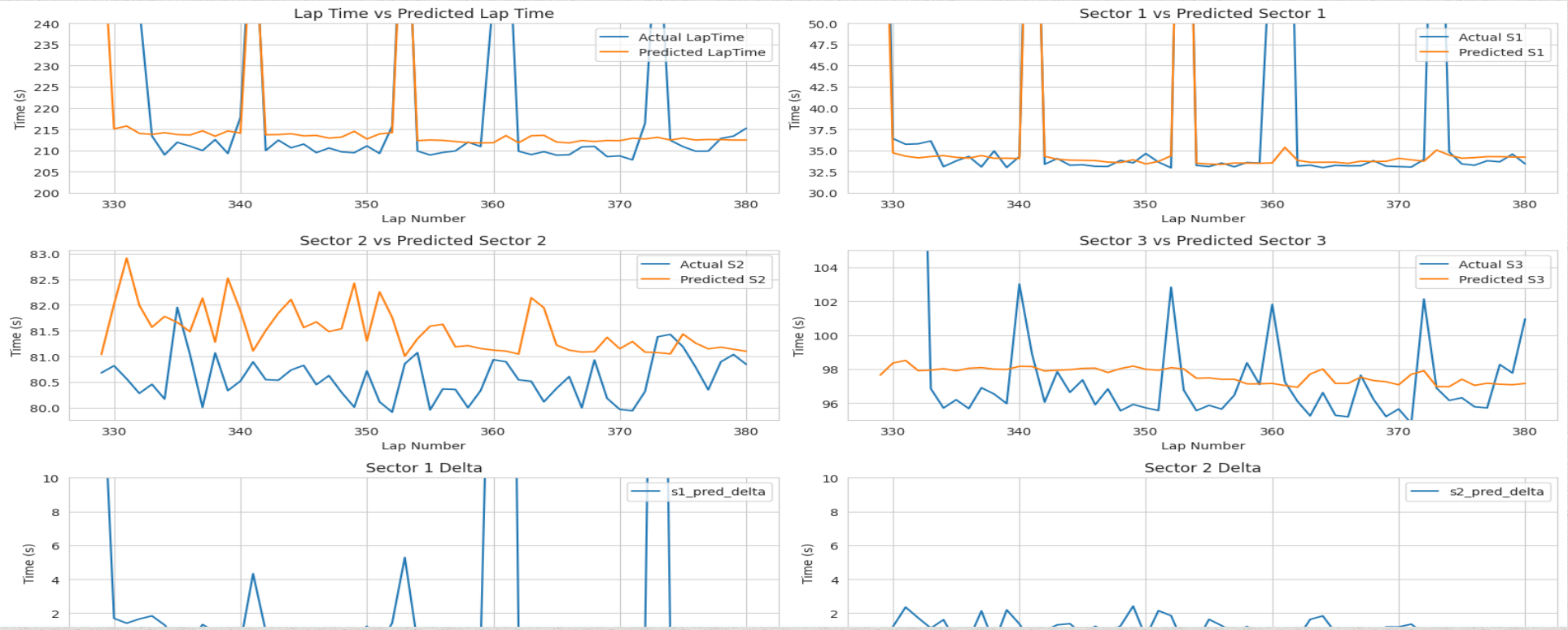
Huberloss delta=1	Sector 1	Sector 2	Sector 3
Simple RNN	0.5700	0.8209	0.2654
LSTM	0.3994	0.1094	0.2425
GRU	0.4506	0.1111	0.1947
Generic Transformer	0.3358	0.2047	0.2333
TSMixer	0.2021	0.1351	0.2069

F1 Bahrain

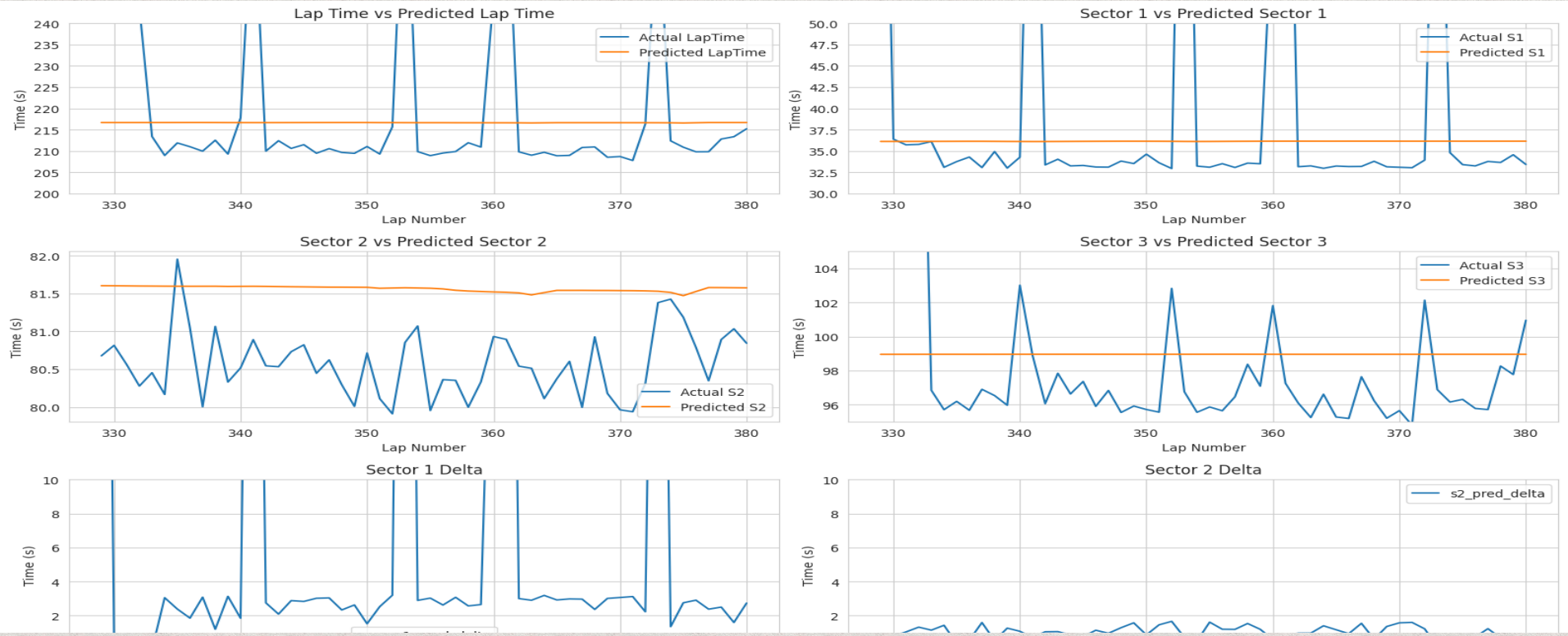
Huberloss delta=1	Sector 1	Sector 2	Sector 3
Simple RNN	0.1824	0.3126	0.2729
LSTM	0.1671	0.2448	0.3269
GRU	0.2083	0.3192	0.3403
Generic Transformer	0.1821	0.2111	0.7275
TSMixer	0.2335	0.4105	0.3428

*One run each, not significant, large variation in training, just for curiosity.

TSMixer Graphs



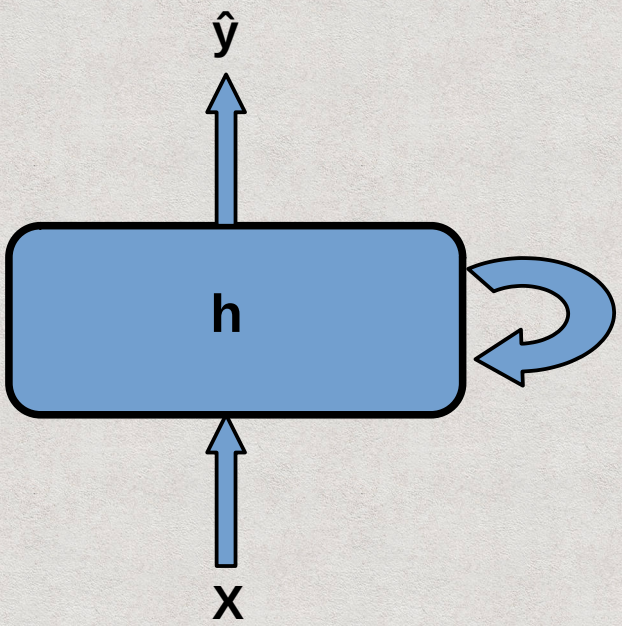
LSTM Graphs


Lap Time vs Predicted Lap Time
Actual LapTime
Predicted LapTime
Time (s)
Lap Number
Sector 1 vs Predicted Sector 1
Actual S1
Predicted S1
Time (s)
Lap Number
Sector 2 vs Predicted Sector 2
Actual S2
Predicted S2
Time (s)
Lap Number
Sector 3 vs Predicted Sector 3
Actual S3
Predicted S3
Time (s)
Lap Number
Sector 1 Delta
Time (s)
Sector 2 Delta
s2_pred_delta
Time (s)

Basic RNN Architecture

Recursive Neural Networks

- What they are:
  - Typical neural networks with a recurring component (time)
  - Have a hidden state that is updated with each recurrence
- Why they are important:
  - They are ‘good’ at predicting future changes in a series.
  - They are the precursor to transformers

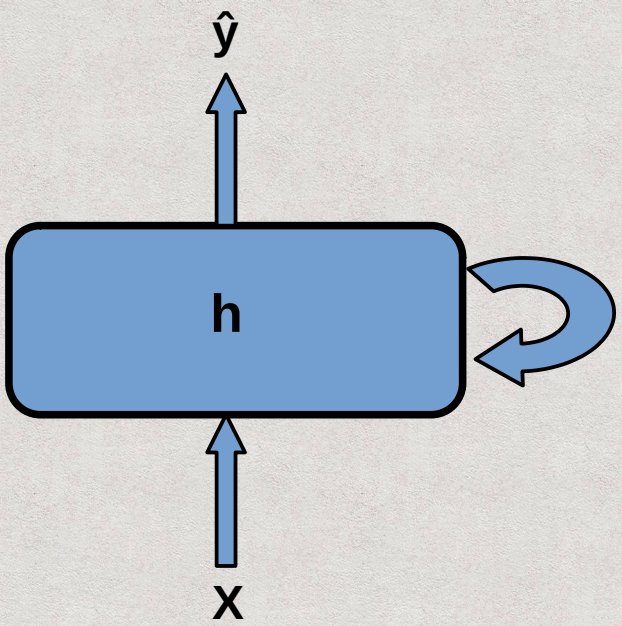


Architecture

- This is a singular recurrent neuron.
- It has has a memory **h** – hidden state
- It has an input **X**
- It has an output **Ŷ**

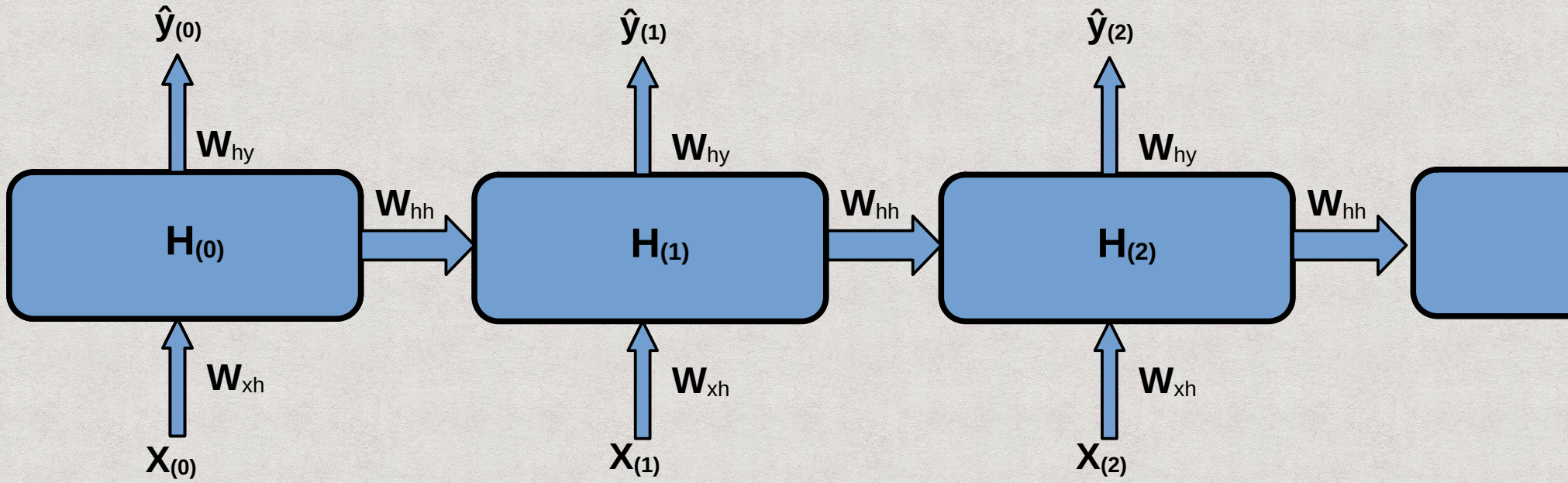
It also repeats, or if you prefer *recurs*

Over t steps starting from 0, updating **h** each time.



Architecture(forward pass)

- To see this more clearly we need to ***"unravel through time"***



Then use $\hat{y}_{(t)}$ to calculate our loss $L_{(t)}$. $L_{(total)} = L_{(0)}+L_{(1)}+L_{(2)}+L_{(3)}+\ldots$

Architecture(Back propagation through time)

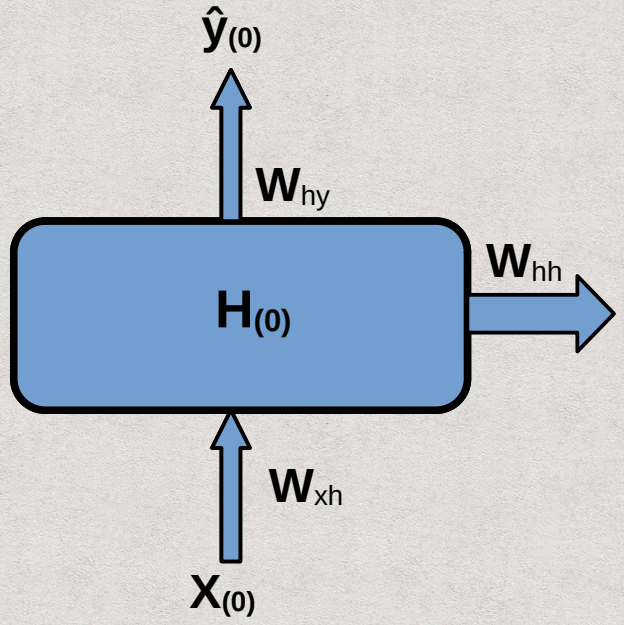
(Maths - feel free to ignore)

Models that use RNNs, have to back-propagate in two different ways

- Through time (through each value of t) (has to be done before moving on to next layer)
- By layer (The normal conventional way)

Lets define some terms:

$$y_t = W_{hy} h_t + b_y$$
$$h_t = tanh(W_{hh} h_{t-1} + W_{hh} x_t + b_h)$$
$$a_t = W_{hh} h_{t-1} + W_{hh} x_t + b_h$$



Architecture(Back propagation through time)

(Maths - feel free to ignore)

Weights get updated by using gradient descent on the gradient of the loss function over our weight.
So lets find gradients

$$y_t = W_{hy} h_t + b_y$$

$$\frac{\partial L}{\partial W_{hy}} = \sum_t \frac{\partial L_t}{\partial W_{hy}}$$

$$\frac{\partial L_t}{\partial W_{hy}} = \frac{\partial L_t}{\partial y_t} \frac{\partial y_t}{\partial W_{hy}}$$

$$\boxed{\frac{\partial L}{\partial W_{hy}} = \sum_t \frac{\partial L}{\partial y_t} h_t}$$

$$\frac{\partial L_t}{\partial W_{xh}} = \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial W_{xh}}$$

$$\frac{\partial L_t}{\partial W_{xh}} = \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial a_t} \frac{\partial a_t}{\partial W_{xh}}$$

$$\boxed{\frac{\partial L}{\partial W_{xh}} = \sum_t \frac{\partial L_t}{\partial h_t} sech^2(a_t) x_t}$$

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial a_t} \frac{\partial a_t}{\partial W_{hh}}$$

$$\boxed{\frac{\partial L}{\partial W_{hh}} = \sum_t \frac{\partial L_t}{\partial h_t} sech^2(a_t) h_{t-1}}$$

Architecture(Back propagation through time)

(Maths - feel free to ignore)

So for h_1 the oldest h state where there are 3 t states for example the impact that h_1 has on the loss function is:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L_1}{\partial h_1} + \frac{\partial L_2}{\partial h_1} + \frac{\partial L_3}{\partial h_1}$$

$$\frac{\partial L}{\partial h_1} = \frac{\partial L_1}{\partial h_1} + \frac{\partial L_2}{\partial h_2}\frac{\partial h_2}{\partial h_1} + \frac{\partial L_3}{\partial h_3}\frac{\partial h_3}{\partial h_2}\frac{\partial h_2}{\partial h_1}$$

Becomes the sum of longer and longer products which if above 1 will eventually explode and if below 1 will vanish